

E02 · RL과 데이터 — 보상으로배우기, 스스로 만든 데이터로 배우기

이 권은 두 질문에 답한다. 첫째, **정답 문자열이 없는 것을 어떻게 가르치는가**. “도움이 되는 답”, “원문에 충실한 요약” 같은 목표에는 대조할정답도 통과시킬 테스트도 없다. 둘째, **가르칠 데이터 자체가 없으면 어떻게 만드는가, 그리고 모델이 자기가 만든 데이터를 먹기 시작하면 무엇이무너지는가**. 앞 질문의 도구가 RL 이고, 뒤 질문의 도구가 synthetic data 다.

두 도구는 언제나 같이 나온다. 데이터 생성이 채점할 대상을 만들고, RL 이그것을 채점한다. 이 권을 읽고 나면 sleep-time compute 논문을 펼쳤을 때**무엇이 상태이고, 무엇이 보상이고, 누가 데이터를 만들었는가** 세질문으로 그 논문을 분해할 수 있다 — 그리고 그 세 질문이 곧 GPU 청구서의세 항목이다.

먼저 이 시리즈의 뼈대 식 하나를 놓는다. 이 권은 이 식 하나의 두 절반을각각 다룬다.

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s) \quad (\text{U-}\Theta)$$

기호 풀이 Θ (세타) = 배포되는 가중치 전체 (slow weights). k = **sleep 라운드** 번호, 곧 한 번의 유티 시간에 도는 학습 사이클하나. η_{Θ} (에타) = 그 라운드의 보폭 (learning rate). $\nabla_{\Theta} \mathcal{L}$ = 손잡이마다 “어느 쪽으로 얼마나돌릴지”를 알려주는 화살표 (gradient). $\mathcal{L}$ = 틀린 정도를나타내는 숫자 하나 (loss). $\mathcal{R}_k$ (캘리그래픽 R) = 그 라운드가쓰는 **학습 집합**. $\mathcal{H}_k = k$ 라운드까지 쌓인 경험. $\text{gen}(\cdot; B_s)$ = 예산 B_s 로 학습 집합을 만드는 절차. B_s = sleep 라운드에 쓸 수 있는 계산 예산 (FLOPs 또는 tokens).

이 식은 이런 뜻. “유티 시간마다 학습 집합을 하나 만들고, 그것으로 배포 가중치를 한 걸음 옮긴다.” 왼쪽 절반은 Θ 를**움직이는** 절차이고, 오른쪽 절반 (gen)은 그 재료를**만드는** 절차다. 1부가 왼쪽, 2부가 오른쪽이다.

주의 이 권에는 **R**이 두 개 나온다. 캘리그래픽 $\mathcal{R}_k$ 는 학습집합이고, 맨 대문자 R 은 **보상** (reward)이다. 원 논문들은보상을 소문자 r 로 쓰지만 이 시리즈는 r 을 LoRA rank에예약했으므로 보상을 R 로 옮겨 쓴다. 이 둘을 섞으면 이 권의 절반이읽히지 않는다.

1부. 보상으로배우기

1. 무엇이 state 이고 무엇이 reward 인가

RL을 처음 만나는 서빙 엔지니어가 막히는 곳은 알고리즘이 아니라**대응**이다. “환경과 상호 작용하며 누적 보상을 최대화한다”는 문장은로봇과 게임을 떠올리게 하는데, LLM에는 로봇도 물리 환경도 없다.

표 1 — LLM 위의 RL 대응. 오른쪽 열은 독자가 매일 튜닝하는 대상이다.

RL 의 항	LLM 에서의 정체	decode 세계의 대응물
state	지금까지의 token 열 = prompt + 생성된 부분	KV cache 가 담고있는 바로 그것
action	다음 token 하나	한 번의 decode step 의 출력
action space	vocabulary 전체	logits 벡터의 차원
전이	state 뒤에 방금 뽑은 token 을 붙이기	KV cache append
episode	한 번의 완결된 생성	한 request 의 decode 전체
horizon	생성 길이	max_new_tokens
reward	생성이 끝난 뒤 밖에서 도착하는 스칼라 하나	대응물없음 — 이것이 새것이다

여기서 셋을 강조한다.

첫째, 전이가 결정론적이고 완전히 알려져 있다. 다음 state 는 지금 state 뒤에 방금 뽑은 token 을 이어 붙인 것, 곧 문자열 concat 이다. 로봇 RL 이 씨름하는 환경 모델 학습이 여기에는 없다. 둘째, 보상은 거의항상 terminal 이다. 중간 token 에는 보상이 없고, 생성이 끝난 뒤 채점기가한 번 호출되어 숫자 하나를 준다. 500 token 을 뽑고 0 점을 받으면 그중 어느결정이 잘못이었는지는 알려주지 않는다. 셋째, episode 가 유한하고 짧으므로 RL 교과서의 discount factor 는 쓰지 않는다.

비유 RL 교과서의 그림은 로봇이 미로를 걷는 장면이다. 여기서 미로는 KV cache 이고, 한 걸음은 token 하나이며, 미로의 구조는 이미 전부 알려져있다. 이 비유가 깨지는 지점 — 로봇 RL 난이도의 절반은 “환경이 어떻게 반응할지 모른다”에서 오는데, LLM 에는 그 모름이 아예없다. 그래서 LLM RL 은 예외적으로 다루기 쉽고, 대신 어려움이 전부보상을 어디서 구할 것인가 한 곳에 몰린다.

시스템 모델링 관점 RL 의 state 가 KV cache 와 같은 것이라면, RL 훈련의 절반은 batch decode job 이다. 후보 생성 단계는 독자가 매일 보는 그 워크로드 — memory-bandwidth bound, KV cache 가 HBM 을 지배, batch size 와 prefix 공유가 처리량을 정하는 — 와 문자 그대로 같다. “RL 로 후처리한다”에서무엇이 GPU 를 먹는지는 절반은 서빙, 절반은 훈련이라고 생각하면 정확하다.

자세한 것은 ch05 § 05.1.

2. policy gradient — 보상은 미분되지 않아도된다

policy 는 state 에서 action 으로의 조건부 분포다. LLM 에는 별도의 policy network 가 없다 — 모델 자신이 policy 다. softmax 출력이 곧 policy 이고, policy 를 갱신한다는 것은 θ 를 움직인다는 것, 즉 식 (U- θ) 를 실행한다는 것이다.

rollout 은 현재 policy 로 prompt 하나에서 끝까지 sampling 한완결된 생성 하나와, 거기 매겨진 보상의 쌍이다. rollout 하나 = decode 한번이다.

기호 풀이 π_θ (파이) = policy, 곧 모델 자신. q = prompt. z = rollout, 곧 생성된 token 열 하나. R = 그 rollout 이 받은 보상 (숫자하나). $\hat{A}$ (A-hat) = advantage, 3 절에서 정의한다. M = 한 prompt 당 뽑는 rollout 개수.

핵심 식은 하나다.

$$\nabla_{\Theta} \mathcal{L}_{RL} = - \mathbb{E}_{z \sim \pi_{\Theta}(\cdot | q)} \left[\hat{A}(z) \nabla_{\Theta} \log \pi_{\Theta}(z | q) \right]$$

이 식은 이런 뜻. “샘플이 좋았으면 그 샘플을 다시 낼 확률을 올리고, 나빴으면 내린다.” 그리고 결정적으로, **gradient** 는 보상 자체를 미분하지 않는다. 미분되는 것은 $\log \pi_{\Theta}$, 곧 “모델이 그문장을 낼 확률” 뿐이다.

이 한 가지 사실이 RL 을 쓰는 유일하고 충분한 이유다. 보상은 미분불가능해도 된다 — 사람이어도, 프로그램이어도, 다른 모델이어도, 심지어 “finetune 한 번 돌려서 평가한 점수” 여도 된다. RL 은 **미분 불가능한 채점기를 loss 로 바꾸는 장치**다. 유도는 ch05 가 한다.

문장 하나의 확률은 그 안의 token 확률들의 곱이므로 로그를 취하면 합이 된다. 그래서 실제 계산에서는 rollout 전체에 매겨진 스칼라 하나가 **그 안의 모든 token 에 똑같은 계수**로 곱해진다. 어느 token 이 잘했는지에 대한 정보는 어디에도 없고, 그 분리는 여러 rollout 에 걸친 통계로만 일어난다. 이것이 credit assignment 문제다.

비유 팀 프로젝트에 성적 하나만 나오는 상황이다. 잘한 사람과 논 사람이 같은 점수를 받는다. 이 비유가 깨지는 지점 — 팀원끼리는 누가 무엇을했는지 서로 알고 증언할 수 있지만, token 들은 서로를 증언하지 못한다. 분리 정보의 유일한 출처가 **반복 표본**뿐이며, 그래서 rollout 개수 M 은 하이퍼파라미터가 아니라 **신호 대 잡음비** 그 자체다.

갱신 단계의 연산 모양은 놀랄 만큼 평범하다. 이미 확정된 token 열 위의 forward + backward 이므로 GEMM shape 이 SFT 와 완전히 같고, 달라지는 것은 sequence 마다 스칼라 하나가 loss 에 곱해진다는 것뿐이다. **RL 의 갱신단계는 가중치가 붙은 SFT** 다.

시스템 모델링 관점 RL 한 iteration 은 arithmetic intensity 가 서로 정반대인 세 단계다. ① **rollout 단계** — decode. memory-bandwidth bound, KV cache 가지 배. ② **채점 단계** — 무엇이 채점하느냐에 따라 GPU forward 일수도, CPU 프로그램 일 수도, 외부 API 일 수도 있다. ③ **갱신 단계** — prefill 모양의 큰 GEMM, compute bound. 실무 스택이 sampler(vLLM 류)와 trainer 를 따로 두고 weight 를 왕복시키는 이유가 이것이다. sleep 라운드 하나에 추론 스택과 훈련 스택이 **동시에** 상주한다는 뜻이고, 클러스터 계획에서 이 사실이 가장 먼저 부딪힌다.

자세한 것은 ch05 § 05.2.

3. advantage — “평소보다 좋았는가” 로 질문바꾸기

앞 식의 $\hat{A}$ 자리에 보상 R 을 그냥 넣으면 두 가지가 망가진다. 분산이 커지고, 더 나쁘게는 모든 rollout 이 양의 보상을 받으면 전부 확률이 올라가 아무것도 구분되지 않는다.

advantage는 그 rollout의 보상에서 같은 prompt에 대한 **기준선** (baseline)을 뺀 값이다. 기준선을 빼도 gradient의 기댓값은 변하지 않고 분산만 줄어든다. advantage가 하는 일은 질문을 바꾸는 것이다 — “이 rollout이 좋았는가”가 아니라 “이 rollout이 **평균보다** 좋았는가”.

기준선을 만드는 방식이 알고리즘 계보를 가른다. 하나는 별도의 **value network**를 학습해 “이 prompt는 보통 몇 점짜리인가”를 예측하는 것 (PPO 계열)이고, 다른 하나는 같은 prompt에서 M 개의 rollout을 뽑아 그 **표본 평균**을 기준선으로 쓰는 것 (group-relative 방식)이다. 후자는 value network를 통째로 없애 HBM에서 모델 하나를 지운다. 알고리즘 선택이 곧 메모리 예산 결정이라는 것이 이 절의 실질이다.

비유 반 평균 대비 점수다. 80점이 잘한 것인지 못한 것인지는 반 평균을 알아야 정해진다. **이 비유가 깨지는 지점** — 반 평균은 고정된 모집단의 성질이지만, 여기서 기준선은 **policy가 움직이면 같이 움직인다**. 모델이 좋아질수록 기준선이 올라가므로, 같은 보상이 라운드마다 다른 advantage를 만든다. 학습이 진행되면 신호가 저절로 줄어드는 구조다.

여기에 안전장치 둘이 붙는다. **KL penalty**는 참조 모델 (보통 RL 시작 시점의 checkpoint)에서 멀어지는 것을 벌해, policy가 보상만 좇다가 언어 모델이기를 그만두는 것을 막는다. **clipping**은 한 batch의 rollout으로 여러 번 가중치를 옮길 때 너무 멀리 가지 않도록 잘라낸다.

주의 첨자 두 개를 여기서 못 박는다. s 는 optimizer가 밟는 **SGD step**이고 k 는 **sleep 라운드**다. 한 라운드가 여러 step을 품으며, 식 $(U-\Theta)$ 의 Θ_{k+1} 은 그 step들이 전부 끝난 뒤의 결과물이다. rollout을 다시 뽑는 주기는 k 의 시계, clipping이 지키는 것은 s 의 시계다. 두 시계를 섞으면 비용 계산이 통째로 틀린다.

자세한 것은 ch05 § 05.2.

4. RLHF — 채점기를 학습해서 사람을 대신세운다

앞 절들은 보상이 이미 있다고 가정했다. 문제는 우리가 원하는 것 대부분에 채점기가 없다는 것이다. RLHF는 이 공백을 **채점기를 학습해서** 메운다.

reward model은 사람이 매긴 선호 비교 데이터로 학습된, (prompt, 응답) 쌍을 점수 하나로 보내는 함수다. 설계의 핵심은 사람에게 절대 점수를 매기게 하지 않고 **둘 중 어느 쪽이 나은가**만 묻는 것이다. 절대 점수는 평가자 간에 재현되지 않지만 쌍 비교는 상당히 재현된다. 그 결과 한가지 성질이 따라온다 — reward model이 배우는 것은 점수의 **차이** 뿐이고, 눈금의 원점은 정의되지 않는다. “이 응답은 7.2 점”이라는 문장에는 의미가 없다.

표 2 — RLHF 세 단계에서 무엇이 움직이는가

단계	움직이는 것	데이터
SFT	Θ	사람이 쓴 (prompt, 응답)
reward model 학습	별도 모델의 파라미터	사람의 쌍 비교
RL	Θ	policy가 뽑은 rollout, 채점은 reward model

1 단계가 먼저인 이유는 실용적이다 — rollout 이 최소한의 형식을 갖추어야 reward model 이 의미 있는 점수를 준다. 3 단계가 RL 이어야 하는 이유는 reward model 을 미분할 수 없어서가 **아니다**. reward model 은 미분가능하다. rollout 을 뽑는 **sampling** 이 미분 불가능하기 때문이다.

비유 reward model 은 사람 채점자를 흉내 내도록 훈련된 조교다. 사람은 느리고비싸지만 조교는 GPU 안에 있어 무한히 부를 수 있다. **이 비유가 깨지는 지점** — 조교는 처음 보는 유형의 답안을 만나면 모르겠다고 말하지만, reward model 은 훈련 분포 밖에서도 자신 있게 점수를 낸다. 이 침묵하지 않는 성질이 7 절 reward hacking 의 뿌리다.

시스템 모델링 관점 3 단계의 HBM 회계를 세면 RLHF 가 왜 무거운지 바로 보인다. 동시에 상주해야 하는 모델이 넷이다 — 학습 중인 policy, KL 항용 frozen 참조모델, frozen reward model, 학습 중인 value network. 넷을 모두 7B 로잡으면 bf16 가 중치만 $4 \times 14 \text{ GB} = 56 \text{ GB}$ 이고, 여기에 학습 대상 둘의 optimizer state 가 더 붙는다 (56 GB 는 자릿수를 잡기 위한 예시 계산이며, 7B 와 bf16 2 bytes/elem 은 이 시리즈가 고른 값이다). **이후 알고리즘 계보 전체가 “이 넷 중 몇 개를 지울 수 있나” 의역사인** 것은 우연이 아니다 — group-relative 기준선이 value network 를 지우고, 다음 절의 DPO 가 sampler 와 reward model 을 함께 지운다.

자세한 것은 ch05 § 05.3.

5. DPO — 채점기와 sampler 를 함께지운다

DPO(Direct Preference Optimization) 는 reward model 을 명시적으로 학습하지 않고, 선호 쌍 자체를 분류 문제로 바꿔 policy 를 직접 갱신하는 방법이다. 출발점은 KL 로 정규화된 RL 목적의 최적해가 닫힌 형태를 갖는다는 관찰이고, 그 관계를 뒤집으면 보상을 policy 로 표현할 수 있어 reward model 이 식에서 통째로 사라진다. 유도는 ch05 가 한다.

DPO 의 loss 에서 중요한 것은 **무엇이 없는지**다. sampling 이 없다. rollout 이 없고, 스칼라 보상이 없고, advantage 가 없고, value network 가 없다. 남은 것은 고정된 선호 쌍 위의 지도 학습뿐이다.

표 3 — 같은 선호 데이터를 쓰는 두 경로의 부품 대조

부품	RLHF (3 단계)	DPO
reward model	학습하고 상주	없음
sampler(rollout 생성)	매 iteration	없음
value network	있음 (또는 group 기준선)	없음
참조 모델	있음 (KL)	있음 (loss 안에)
워크로드 모양	decode + 채점 + 훈련	훈련만

대가는 **off-policy** 라는 것이다. 선호 쌍이 현재 policy 의 분포에서 오지 않으므로, policy 가 데이터를 만든 분포에서 멀어질수록 신호가 낡는다. RLHF 의 rollout 은 정의상 언제나 현재 policy 의 것이고, DPO 의 쌍은 한 번 고정된다.

비유 RLHF는 실시간 대국에 코치가 붙어 매 수를 평가하는 것이고, DPO는 남이 둔 대국 기보를 보며 “이 수가 저 수보다 낫다”를 외우는 것이다. **이비유가 깨지는 지점** — 기보는 실력이 늘어도 그대로지만 학습자는 변한다. 초보 때 유용하던 기보가 실력이 올라가면 자기 수준과 무관한 국면만 담게된다. 이것이 off-policy의 정확한 내용이며, DPO의 품질 상한이 데이터 갱신주기에 걸리는 이유다.

이 시리즈는 두 방법에 우열을 매기지 않는다. 선택을 정하는 변수는 목적도메인과 선호 데이터의 갱신 주기다. 다만 sleep 라운드 관점의 요점은하나다 — **DPO의 워크로드 모양은 SFT와 같다**. 고정 배치, sampler 없음, 채점기 없음. 유틸 시간에 엮기가 비교할 수 없이 싸고 스케줄링이훈련 job 하나로 끝난다.

자세한 것은 ch05 § 05.4.

6. RLVR — 채점기를 프로그램으로바꾼다

RLVR(Reinforcement Learning with Verifiable Rewards)은 보상을사람도 학습된 모델도 아닌 **결정론적 검증 프로그램**이 산출하는 RL이다. 정답 문자열 대조, 단위 테스트 실행, 파서·컴파일러통과 여부, 출력 형식 정규식이 전형적인 검증기다. 식은 2절 그대로이고 R 의 출처만 바뀐다. 바뀌는 것은 전부 **회계와 실패 모드**다.

표 4 — 검증기의 네 종류

검증기	예	결정론적인가	어디서 도는가
문자열 대조	수학 정답, 추출형 QA	완전	CPU, 무시 가능
실행	단위 테스트, 컴파일	완전 (샌드박스 고정 시)	격리 프로세스
형식 검사	JSON 파싱, 정규식, 스키마	완전	CPU
모델 채점	LLM-as-a-Judge, 루브릭	아님	GPU 또는 API

앞의 셋만이 엄밀한 의미의 RLVR다. 네 번째 줄은 “검증기”라는 이름을 쓰되실제로는 학습된 채점기이므로 4절 조교 비유의 약점을 그대로 물려받는다.

검증기가 프로그램일 때 얻는 것은 셋이다. 사람 라벨이 필요 없고, 채점기가학습된 함수가 아니므로 policy가 그 일반화 오차를 타고 오를 수 없으며, **보상을 무인으로 임의 횟수 거의 공짜로 만들 수 있다**. 이 시리즈의관심은 세 번째에 있다. sleep 라운드는 정의상 사람이 없는 시간이고, 신호를 사람이 준다면 라운드 처리량이 사람의 처리량에 묶여 유틸 시간을쓴다는 전제 자체가 무너지기 때문이다. 한계도 같은 자리에서 나온다 — **검증 가능한 것은 전체의 일부**다. 임의의 문서를 가중치에 넣었는지판정하려면 그 문서에 대한 문제와 답이 있어야 한다.

비유 단위 테스트다. 통과/실패가 기계적으로, 무한히, 공짜에가깝게 나온다. **이 비유가 깨지는 지점** — 코드에는 테스트를 쓸수 있지만 “좋은 요약”에는 쓸 수 없다. 그리고 테스트를 통과시키는방법과 올바른 코드를 쓰는 방법이 같다는 보장은 어디에도 없다. 후자가 7절의 주제다.

시스템 모델링 관점 검증기는 GPU 밖에 산다. 샌드박스, 테스트 러너, 파서는 CPU·IO 서비스이고, 결정론적이므로 캐시할 수 있고 batch 로돌릴 수 있으며 실패해도 재시도가 안전하다. RL 루프의 tail latency 가 GPU 에서 그 서비스로 옮겨간다. reward model 방식 (rollout 마다 채점 모델 forward 가 한 번 더) 과 비교하면 비용이 완전히 다른 자리에 선다 — 한쪽은 HBM 과 GEMM, 다른 쪽은 프로세스 격리와 타임아웃 정책이다.

자세한 것은 ch05 § 05.5.

7. reward hacking — 보상이 목적을 대신할때

reward hacking 은 policy 가 의도한 목적을 달성하지 않은 채 보상신호의 결함을 최대화하는 현상이다. 버그가 아니라 **구조적귀결**이다 — 보상은 목적의 대리이고, RL 은 그 대리를 최대화하도록 설계된 최적화기다. 대리과 목적이 갈라지는 지점을 policy 가 찾아내는 것은 실패가 아니라 정확히 시킨 일이다.

두 종류로 나누면 대응책이 달라진다. **학습된 채점기의 오차를 타는 것** — reward model 은 고정된 분포에서 학습되었는데 policy 는 학습하면서 분포를 옮긴다. 점수는 계속 오르는 데 실제 품질은 꺾인다. 대응은 KL 향으로 참조 모델 근처에 묶기, 새 분포에서 채점기 재학습, ensemble 로 불확실성별하기다. **결정론적 채점기의 허점을 타는 것** — 테스트만 통과시키는 코드, 형식만 맞춘 빈 답. RLVR 도 면역은 아니지만 이쪽은 **재현 가능**해서 찾아 고치면 사라진다.

가장 흔한 형태에는 이름을 붙일 만하다. **길이 인플레이션** — 길수록 점수가 높아지는 채점기 (사람 평가자를 포함해) 에서 policy 는 길어진다. 이 corpus 안에 문서화된 사례가 있다. SEAL 은 RL 이 진행될수록 자기가 만든 학습 문서가 길어지는 것을 관측하고, 길이만 따로 조작해 보는 실험을 실제로 한다. RL 을 **한 라운드도 돌리지 않고** prompt 만 “implications-long” 으로 바꾸면 49.3, “rewrite” 로 바꾸면 49.4 이며, 두 값 모두 2 라운드 RL 을 돌린 본문 헤드라인 47.0 보다 높다 [SEAL Table 10]. 프롬프트 한 줄이 RL 루프 전체보다 크다.

핵심 이 표가 보여주는 것은 reward hacking 의 교과서적 **진단 절차**다 — 보상과 상관된 표면 변량 (여기서는 길이) 을 의심하고, RL 없이 그 변량만 prompt 로 직접 조작해 같은 이득이 나오는지 본다. 이 시리즈가 못 박는 규범은 그쪽이다. **RL 이득을 보고할 때는 이 통제군이 있어야 하고, 없으면 그 이득은 기체에 귀속될 수 없다.**

비유 속도계를 조작해 연비 기록을 올리는 것이다. **이 비유가 깨지는 지점** — 조작된 속도계는 찾아서 고치면 끝나지만, 학습된 채점기의 오차는 한 곳에 있지 않다. 고칠 부품이 없고 분포 전체가 조금씩 어긋나 있으므로, 대응이 수리가 아니라 **제약** (KL 향) 일 수 밖에 없다.

길이 인플레이션은 품질 지표만 흐리는 것이 아니라 **회계를 망가뜨린다**. rollout 이 길어지면 decode 시간이 선형으로 늘고 KV cache 가 선형으로 늘며, 채점기가 LLM 이면 채점 비용까지 함께 늘어난다. 즉 sleep 예산 B_s 가 라운드마다 커진다. 학습 초기에 계산한 손익 분기는 나중 라운드에서 성립하지 않는다.

자세한 것은 ch05 § 05.6.

8. 손으로 따라가기 — 한 위치의 확률이 어디로가는가

숫자 몇 개로 1 부 전체를 확인한다. 아래 설정과 값은 전부 이 시리즈가 만든 예시 계산이다.

설정. prompt 하나에 rollout $M = 4$ 개를 뽑았고 보상은 이진이다. 결과는 $(1, 0, 1, 0)$. 어떤 한 decode 위치에서 후보 token 이 둘만 살아있고, 확률은 각각 0.25와 0.75다. rollout 은 그 위치에서 확률 0.25 짜리를 뽑았다. 보폭은 $\eta_{\Theta} = 0.1$.

group-relative 방식은 같은 prompt 의 표본 평균을 기준선으로 쓴다. 네보상의 평균 이 0.5 이므로 계수는 $(+1, -1, +1, -1)$ 이 된다. **음수가 있다.** SEAL 이 채택한 ReST^{EM} 계열은 기준선을 빼지않고 보상을 그대로 계수로 쓴다. 계수는 $(1, 0, 1, 0)$ 이다. **음수가 없다.**

표 5 — 같은 rollout, 같은 위치, 다른 알고리즘 (그 위치 token 의 확률변화)

그 rollout 이 받은 보상	group-relative 계수	ReST^{EM} 계수	새 확률 (group)	새 확률 (ReST^{EM})
$R = 1$	+1	1	0.279	0.279
$R = 0$	-1	0	0.223	0.250 (불변)

두 알고리즘은 성공한 rollout 에 대해 정확히 같은 일을 하고, 실패한 rollout 에 대해 완전히 다른 일을 한다. **ReST^{EM} 은 실패를 벌하지 않고 그냥 버린다.** 확률을 내리는 방향의 gradient 가 존재하지않는다는 뜻이며, 이름 그대로 “보상 1 을 받은 것들만 모아 SFT” 다.

퇴화 지점이 두 방식에서 서로 다르다. 같은 prompt 의 rollout 4 개가 전부 정답이면, group-relative 에서는 평균과의 편차가 0 이므로 계수가 전부 0 — **gradient 가 정확히 0 이고 아무것도 배우지 않는다.** 실무에서 “전부 맞거나 전부 틀리는” 문제를 난이도로 걸러 빼는 이유가 이것이다. 반면 ReST^{EM} 에서는 계수가 $(1, 1, 1, 1)$ 이 되어 넷 다 확률이 올라간다 — 자기 샘플 위의 평범한 SFT 로 퇴화한다.

마지막으로 이 한 prompt 의 채점 비용을 센다. SEAL 의 채점기는 후보 하나 평가에 30 – 45 초가 걸리므로 [SEAL § 5], $M = 4$ 면 gradient 를 한 번도 계산하기 전에 2 – 3 분이 소비된다. 그중 절반 (보상 0 인 둘) 은 계수 0 을 받아 갱신에 전혀 기여하지 않는다 — **채점 비용의 50%가 버려진다.**

자세한 것은 ch05 § 05.2 와 ch05 § 05.7.

9. 진짜 RL 과 어휘만 빌린 RL

이 corpus 의 두 논문이 RL 을 정반대로 사용한다. 나란히 놓으면 1 부의 어휘가 무엇을 판별하는 도구인지 드러난다.

SEAL 의 outer loop 는 진짜 RL 이다. 다만 PPO 도 group-relative 도 아니다. 논문은 “GRPO 와 PPO 같은 다양한 on-policy 방법을 실험했으나 학습이 불안정했다” 고 쓰고 [SEAL § 3.1], 대신 rejection sampling + SFT 에 해당하는 더 단순한 방법을 택한다. 세 향의 대응도 표준형과 다르다. **action 이 token 하나가 아니라 학습 문서 한 편 전체이고, reward 는 그 문서로 실제 finetune 을 한 번 돌려 얻은 모델을 downstream 과제에서 채점한 점수다.**

세 번째가 이 corpus의 핵심 장치다. 채점기가 프로그램도 사람도 reward model도 아니고 **학습 실행 그 자체**다 — finetune 한 번 + 평가 한번이 보상 함수의 본체다. 표 4의 어느 한 줄에도 깨끗이 들어가지 않으며, 정확히는 뼈대가 두 번째 줄 (실행) 이되 실행의 마지막 채점이 네 번째 줄 (모델 채점) 이다. 따라서 RLVR의 “허점이 재현 가능하다” 는 이점은 여기에 적용되지 않는다.

ReasoningBank은 반대다. agent를 policy 표기로 쓰고 자기 루프가 RL의 학습 동역학을 닮았다고 서술하는데 [ReasoningBank § 3.1, § 5], gradient가 한 번도 계산되지 않는다. backbone은 frozen 체크포인트이고 갱신되는 것은 JSON 파일 하나다. rollout도 있고 자동 채점기도 있지만 policy 갱신 자리에 파일 쓰기가 들어가 있다. 그 자동 채점기의 정확도는 ground truth 대비 72.7%로 측정되어 있다 [ReasoningBank § 5] — 표 4의 넷째 줄 (모델 채점) 이 실제로 얼마나 믿을 만한지에 대해 이 corpus에서 유일하게 재어진 값이다.

핵심 판별 기준은 단순하다 — $\eta_{\Theta} \nabla_{\Theta}$ 가 어디에선가 실제로 실행되는가. ReasoningBank에서는 실행되지 않는다. 결함이라는 뜻이 아니다. 그렇게 작동하도록 설계된 것이고 그이점 (체크포인트 관리도, optimizer state도, 망각 대응도 필요 없다) 은 실재한다. 결함은 두 개의 다른 기제를 같은 단어로 부르는 데서 생긴다.

시스템 모델링 관점 보상의 종류가 곧 sleep 라운드가 먹는 하드웨어의 종류다. SEAL의 한 라운드는 $50 \text{ passage} \times 5 \text{ completion} \times 3 \text{ seed} = 750$ 회의 finetune 실행이고 $2 \times H100$ 에서 약 6 시간이다 [SEAL § B.5]. 반대편 자릿수도 재어져 있다 — ReasoningBank의 채점 + 요약은 과제당 3748.4 tokens 이고, 그 과제총 토큰의 7% 남짓이다 [ReasoningBank Table 5]. 한쪽은 라운드마다 finetune 750 번, 다른 쪽은 과제마다 추론 호출 두 번이다. 같은 단어로 불리는 두 시스템의 sleep 예산이 자릿수 단위로 갈린다.

자세한 것은 ch05 § 05.7.

2부. 스스로 만든 데이터로 배우기

10. 학습 집합은 어디서 오는가

식 (U, Θ) 에는 아직 열지 않은 상자가 하나 있다. $\text{gen}(\mathcal{H}_k; B_s)$ 다. 1부는 학습 집합이 있다고 가정하고 Θ 를 옮기는 법을 다뤘다. 2부는 그 학습 집합을 만드는 절차만 다룬다.

세 이름을 정의한다. **data augmentation**은 이미 가진 데이터 항목에 **라벨을 보존하는 변환**을 적용해 항목 수를 늘리는 절차다. 원항목이 반드시 있어야 한다. **synthetic data**는 사람이 쓰지 않고 **모델이 생성한** 학습 데이터다. 원 항목이 없어도 된다. **self-generated data**는 synthetic data 중에서 **생성기가 지금 학습받는 모델 자신인** 것이다.

셋은 포함 관계가 아니라 축이 다르다. augmentation은 “원 항목이 있는가” 를 묻고, synthetic은 “사람이 썼는가” 를 묻고, self-generated는 “생성기가 누구인가” 를 묻는다.

표 6 — 학습 집합의 출처

출처	생성기	루프가 닫히는가	B_s 가 붙는 곳
사람이 쓴 데이터	사람	아니오	없음
data augmentation (고전형)	결정론적 변환	아니오	CPU dataloader
synthetic data (타 모델)	다른 모델	아니오	타 모델의 decode
self-generated data	직전 라운드의 자기 자신	예	자기 모델의 decode

마지막 줄만 루프가 닫힌다. Θ_{k-1} 이 만든 데이터로 Θ_k 를 만들고, Θ_k 가 만든 데이터로 Θ_{k+1} 을 만든다. 이 되먹임이 2부 나머지의 주제다.

자세한 것은 ch06 § 06.1.

11. LLM에서 augmentation은 decode이고, 형식이 생성기보다먼저다

augmentation의 정의는 vision에서 왔고, 그 정의의 두 조건 중 하나가 text에서 깨진다. 깨지는 쪽은 “라벨을 보존하는 변환”이다. 고양이 사진은 좌우로 뒤집어도 고양이지만, 문장은 동의어를 바꾸면 의미가 바뀌고 순서를 바꾸면 논증이 무너진다. **text에는 라벨을 보존하는 변환군이 알려져있지 않다.**

그래서 LLM 실무의 augmentation은 결정론적 변환을 포기하고 **모델에게 다시 쓰게 하는 것으로** 대체되었다. 그 순간 표 6의 둘째줄과 나머지 줄의 경계가 사라진다 — “원 항목이 있는 augmentation”과 “원 항목을 조건으로 받은 synthetic data”가 구현상 같아진다. 셋에 동시에 걸치면 이 시리즈는 **self-generated로 분류한다**. 분류의 목적이 이름이 아니라 위험 판정이고, 14절의 붕괴 경계는 “생성기가 누구인가”에만 걸리기 때문이다. 원 항목이 있다는 사실은 안전을 조금도 보장하지 않는다.

비유 사진 뒤집기가 문장 바꿔쓰기로 바뀌었다. **이 비유가 깨지는 지점**— 뒤집기는 공짜이고 CPU에서 돌지만, 바꿔쓰기는 **decode**다. 비용이 dataloader에서 GPU로 통째로 옮겨왔고, 그래서 B_s 의 대부분을 차지한다. 문서 1건당 후보 M 개를 뽑는 파이프라인이면 생성 비용은 그대로 M 배다.

여기서 이 corpus가 실제로 켄 숫자가 결정적이다. 같은 passage 집합, 같은 모델을 놓고 데이터 만드는 법만 바꾼 결과다 [SEAL Table 2].

표 7 — 생성기에 따른 no-context SQuAD 정확도 (%) ([SEAL Table 2] 그대로)

방법	passage 1 건 (LoRA)	200 건 CPT	2067 건 CPT
Base model	32.7	32.7	29.0
원 passage로 학습	33.5	36.0	31.2
자기 생성 (RL 없음)	39.7	50.6	43.4
GPT-4.1 synthetic	46.3	59.4	49.2
SEAL(RL로 학습된 자기 생성)	47.0	58.2	46.4

첫 열을 세 증분으로 쪼개면 이 절의 지침이 나온다. 원문을 그대로 학습시키는 것은 거의 아무 일도 하지 않는다 ($32.7 \rightarrow 33.5$). 데이터의 **형식**을 원문에서 함의 목록으로 바꾸면 +6.2

점 (33.5 → 39.7), **생성기 품질**을 올리면 +6.6 점 (39.7 → 46.3), **생성 정책을 RL로 학습**하면 +0.7 점 (46.3 → 47.0) 이다. 세 항의 크기 순서가 예산 배분의 순서다. 마지막 항부터 손대는 것은 순서가 틀렸다.

주의 47.0 이 GPT-4.1 을 앞선다는 사실에는 범위 조건이 붙는다. **그 역전은 세 열 중 한 열에서만 성립한다.** 나머지 두 열에서는 GPT-4.1 이 각각 59.4 대 58.2, 49.2 대 46.4 로 이긴다 [SEAL Table 2, § 4.2]. 이 조건 없이 47.0 을 인용하는 것은 결함이다.

자세한 것은 ch06 § 06.2 와 ch06 § 06.3.

12. 필터는 생성 절차의 나머지 절반이다

생성만으로 학습 집합이 정해진다고 보는 것은 실제 파이프라인을 잘못 그린 것이다. **데이터 품질 필터링**은 생성된 후보 집합에서 실제로 학습에 넣을 부분 집합을 고르는 절차이며, 생성 절차를 후보 생성과 채택 판정 두 단계로 쪼갬다. 판정 함수를 ϕ (파이)라 쓴다.

ϕ 는 세 종류다. **규칙·휴리스틱 필터**(중복제거, 길이 범위, 형식 검사)는 비용이 거의 0 이고 판정이 결정론적이다. **검증기 기반 필터**(과제 점수, 실행 결과, 보상 모델)는 정확하지만 비용이 후보마다 붙는다. **사람 필터**는 가장 정확하고 가장 비싸며 유희 시간에 상각되지 않는다.

SEAL 의 ϕ 는 두 번째의 극단이다 — 후보로 실제 가중치를 갱신하고, 갱신된 모델로 문제를 풀어, 점수가 오르면 채택한다. 그 ϕ 가 얼마짜리인지도 같은 논문이 켜다. 학습-후-평가 보상을 GPT-4.1 루브릭 채점으로 바꾸면 47.0 에서 45.6 으로 1.4 점 떨어지고 시간은 약 6 시간에서 약 5 분이 된다 [SEAL Table 9]. **1.4 점에 약 72 배다.** 필터의 비용-품질 곡선이 매우 평평하다는 뜻이고, 예산이 제약인 시스템에서 비싼 ϕ 를 고를 근거는 그 논문 안에 없다.

비유 필터는 체다. 굵은 것과 가는 것을 갈라낸다. **이 비유가 깨지는 지점** — 체는 있는 것 중에서 고를 뿐, 없는 것을 만들지 못한다. 같은 논문에서 프롬프트를 주지 않고 모델이 스스로 형식을 정하게 한 조건은 13.8 에서 출발해 RL 2 라운드 뒤에도 18.9 다 [SEAL Table 10]. 필터는 분포를 자르는 연산이지 옮기는 연산이 아니다. 소스가 나쁘면 구제하지 못한다.

시스템 모델링 관점 필터는 B_s 의 곱셈 인자다. 후보 M 개를 뽑아 1 개를 채택하면 생성 예산이 M 배이고, ϕ 가 검증기 기반이면 여기에 후보당 판정 비용이 더 붙는다. SEAL 식 ϕ 에서는 그 판정이 finetune 한 번 + 평가 한 번이므로 $B_s \approx M \times (\text{decode} + \text{SFT} + \text{eval})$ 이된다. 상각 계산에 들어가는 것은 이 값이지 decode 항 하나가 아니다. sleep 라운드 비용을 견적낼 때 가장 흔한 과소평가가 여기서 나온다.

자세한 것은 ch06 § 06.4.

13. model collapse — 이름 하나에 현상넷

model collapse는 모델이 자기 계보가 생성한 데이터로 반복 학습될때 성능이 라운드가 갈수록 나빠지는 현상이다. 이 corpus에서 그 경계를유일하게 닫힌 형식으로 가진 논문 (이하 [MCI])의 조작적 정의는 더 좁고날카롭다 — 붕괴를 피한다는 것은 반복에 대해 오차가 **유계로 남는것이다** [MCI § 2]. 이 정의 아래에서는 오차가 늘어나도 수렴하면 붕괴가아니고, 아무리 천천히 늘어도 발산하면 붕괴다.

같은 논문이 이 이름의 남용을 경고하며 네 현상을 분리한다.

표 8 — model collapse라는 이름이 가리키는 네 현상 ([MCI § 4] 분류)

#	현상	무엇이 관측되는가
(0)	test error 의 무계 폭증	held-out loss 가 라운드마다 오르고수렴하지 않는다
(1)	modal collapse	생성 분포가 한두 개 mode 로 수축한다
(2)	uniformity 로의 붕괴	생성 분포가 특징 없이 평평해진다
(3)	artifact 증폭	앞선 합성 데이터의 결함이 라운드마다 커진다

네 항목은 서로를 함의하지 않는다. loss가 유계로 남으면서 mode가 사라질수 있고, 그 반대도 가능하다. 그래서 이 시리즈는 “**붕괴한다**” 를번호 없이 쓰지 않는다.

주의 프로덕션에서 관측 가능한 것은 대개 (0) 뿐이다. held-out perplexity 는대시보드에 있지만 생성 분포의 mode 수는 없다. (1) 과 (3) 은 **알람이울리지 않는 붕괴다**. 17절에서 이 함정으로 돌아온다.

자세한 것은 ch06 § 06.5.

14. accumulate vs replace — 스케줄이 수렴을정한다

루프의 안정성은 학습 규칙이 아니라 학습 집합을 구성하는 스케줄이정한다. 이것이 [MCI]의 결과이자 2 부의 핵심 명제다.

replace는 라운드마다 이전 학습 집합을 버리고 새로 생성한 것으로바꾸는 스케줄이다. **accumulate**는 이전 것을 버리지 않고 새생성분을 덧붙이는 스케줄이다 — 라운드 1의 실패이터가 영원히 남는다. 데이터 양만 accumulate와 맞춘 통제군으로 **replace-multiple**(실데이터는 버리되 합성 코퍼스만 키우기)이 하나더 있다.

주의 식 $(U-\Theta)$ 는 스케줄을 지정하지 않았고, 지정하지 않은 채로 읽으면글자 그대로 **replace**다. 표준형을 그대로 반복하는 시스템은 [MCI]가 발산한다고 증명한 바로 그 레짐에 있다. 스케줄은 세부가아니라 η_Θ 와 나란한 1 급 파라미터다.

기제는 한 줄로 말할 수 있다. accumulate에서 라운드 i 에주입된 잡음은 계수 $1/i$ 로 파라미터에 도달한다 — 그 데이터가 코퍼스에들어갔을 때 i 개 블록 중 하나였기 때문이다. 오

차는 제공이므로 기여가 $1/i^2$ 이 되고, $\sum 1/i^2$ 이 수렴하므로 총합이 유계로 남는다. 결론이 세 스케줄의 라운드당 열화율 ρ_k 로 정리된다 (ρ = 라운드당 성능 손실).

$$\rho_k \propto \begin{cases} 1 & \text{replace} \\ 1/k & \text{replace-multiple} \\ 1/k^2 & \text{accumulate} \end{cases}$$

이 식은 이런 뜻. “반복되는 sleep 루프는 $\sum_k \rho_k$ 가 수렴할 때에만 지속 가능하다.” replace 는 상수를 무한히 더하므로 발산하고, replace-multiple 은 조화 급수라 $\log k$ 로 발산하며, accumulate 만 수렴한다. accumulate 를 영원히 돌린 총 열화는 한 라운드 별점의 $\pi^2/6 \approx 1.6449$ 배를 넘지 않는다 [MCI Eq. 3].

데이터를 더 만드는 것은 기제가 아니다. 실데이터를 계속 보유하는 것이 기제다. replace-multiple 이 그 증거다 — 데이터를 accumulate 만큼 늘리고도 붕괴한다.

실측도 같은 방향이다. TinyStories 위 GPT-2 9M 의 held-out cross-entropy 는 라운드 1 에서 1.82 이고, 라운드 4 에서 accumulate 1.74 · replace 2.39, 라운드 10 에서 replace 2.91 이다. 데이터 양을 맞춘 replace-multiple 도 라운드 4 에서 2.18 로 나빠진다. 디코딩 온도를 0.3 으로 낮추면 격차가 커진다 — replace 는 라운드 10 에서 9.85 로 무너지고 accumulate 는 1.75 로 사실상 불변이다 [MCI Table 2]. 낮은 온도가 더 나쁘다. 그 리고 프로덕션 기본값인 온도 0 은 검증되지 않았다.

비유 복사본을 복사하는 것이다. replace 는 원본을 버리고 복사본만 남기며, accumulate 는 원본을 계속 보관하고 복사본을 옆에 쌓는다. **이 비유가 깨지는 지점** — 복사기의 열화는 단조롭게 흐려지는 방향이지만, 여기서 잡음은 확률적이고 부호가 없다. replace 는 흐려지는 것이 아니라 **표류한다** (random walk). 방향이 정해져 있지 않으므로 “어느 쪽으로 나빠지는지” 보고 잡아낼 수 없다.

자세한 것은 ch06 § 06.6.

15. 손으로 따라가기 — 1 차원으로 세 스케줄 재보기

암산으로 재현할 수 있는 최소 설정을 만든다. 이 절의 숫자는 전부 이시리즈의 산술이다.

설정. 진짜 분포는 평균 0, 분산 1 인 정규분포다. 모델은 파라미터가 평균 하나뿐이고, 학습은 학습 집합의 표본평균을 취하는 것, 생성은 그 평균을 중심으로 뽑는 것이다. 라운드당 표본은 100 개이고 라운드 1 은 실데이터로 시작한다. 표본 100 개 평균의 분산이 $1/100 = 0.01$ 이므로 **이 0.01 이 한 라운드 별점이고 이후 모든 숫자가 그 배수다.**

replace 에서는 라운드 k 가 라운드 $k - 1$ 의 평균 주위에서만 뽑으므로 별점이 그대로 쌓인다 — 오차가 $0.01k$ 인 random walk 다. accumulate 에서는 라운드 i 의 잡음이 $1/i$ 로 섞이므로 오차가 $0.01 \sum 1/i^2$ 이고, replace-multiple 은 그 중간인 $0.01 \sum 1/i$ 이다.

표 9 — 세 스케줄의 test error (한 라운드 별점 0.01 기준, 이시리즈의 산술)

라운드 k	replace	replace-multiple	accumulate
1	0.0100	0.0100	0.0100
5	0.0500	0.02283	0.01464
10	0.1000	0.02929	0.01550
100	1.0000	0.05187	0.01635
∞	발산	발산	0.016449

$k = 100$ 줄이 요점이다. replace 의 오차 1.00 은 **참분포의 분산과 같다** — 100 라운드를 돌고 나면 모델의 추정이 실패본하나보다 나을 것이 없다. 같은 라운드에서 accumulate 는 0.01635 이고 영원히 0.016449 를 넘지 않는다. **약 61 배 차이가 학습 규칙이 아니라 스케줄 하나에서 나온다.**

질문을 하나 더 던지면 가장 반직관적인 사실이 나온다. 코퍼스 크기를고정한 채 매 라운드 **실데이터 비율을 λ (람다) 로유지**하면 어디로 가는가. 답은 유계다 — λ 가 0 보다 크기만 하면오차가 어딘가에 멈추고, 정확히 0 일 때만 random walk 가 된다. $\lambda = 0.5$ 면 0.01333, $\lambda = 0.1$ 이면 0.05263, $\lambda = 0.01$ 이면 0.50251 이다.

그런데 accumulate 에서 실데이터 비율은 $1/k$ 로 **0 을 향해사라진다**. 그런데도 오차는 0.016449 에 멈추고, 그 값은실데이터 비율을 영구히 약 0.37 로 고정한 것과 같은 자리다. **중요한 것은 실데이터 비율이 0 으로 가느냐가 아니라 얼마나 빨리가느냐다.**

두 처방의 가격표는 전혀 다르고, 이것이 실제 선택지다. accumulate 는유계성을 계산으로 산다 (코퍼스가 라운드마다 커진다). 고정 λ 는 유계성을 **새 실데이터 공급**으로 산다 (라운드마다신선한 사람 데이터가 계속 들어와야 한다). 배포된 시스템은 대부분 후자에있고, [MCI] 는 그 레짐을 첫 번째 open question 으로 남긴다 [MCI § 4].

자세한 것은 ch06 § 06.6.

16. 이 경계를 우리 시스템에 옮길 수있는가

옮길 수 없다. $\pi^2/6$ 은 안전 증서가 아니다. 그 경계는 네전제 위에 서 있고, 넷 다 실제 sleep-time 시스템에서 깨진다.

- (i) **라운드마다 모델을 처음부터 다시 학습한다** — 그래서 이론에 η_{Θ} 가 아예 등장하지 않고, optimizer state 도 checkpoint 연속성도 catastrophic forgetting 도 모델링 되지 않는다. (ii) **입력이 동결되어 있다** — 누적된 데이터는 같은 입력을 k 번썩은 것이고 타깃만 재생성된다. 즉 경계가 다루는 것은 누적된**라벨 잡음**이지 입력 분포의 이동이 아니다. (iii) 모델이 참함수를 표현할 수 있고 정칙화가 없다. (iv) **생성에 필터가 없다**— 검증도 보상도 중복 제거도 사람 검토도 없다.

SEAL 은 살아 있는 checkpoint 를 증분 편집하고, 보상으로 후보를 고르며, 입력을 매 라운드 새로 만든다. 네 전제가 전부 깨진다. 기제 자체가 옮겨가지 않는다 — “라운드 i 의 데이터가 코퍼스의 $1/i$ 이므로 잡음이 $1/i$ 로 들어온다” 는 논증은 **이전 데이터가 아니라 이전 파라미터가 운반자일 때 대응물이 없다**. 따라서 이 정리는 그 시스템들을보증하지도, 반증하지도 않는다.

반대 방향으로도 한 칸을 깎아야 한다. 그 논문은 자기 설정을 “최대로비관적” 이라 부르지만, accumulate 에서 원 실데이터는 규정상 모든 라운드에절대량 그대로 남는다. **보호**

성분을 제거할 수 없게 만들어 둔설정은 최악이 아니라 유리한 구성이다. 인터넷 규모의 안심 근거로 옮겨 쓸수 없는 이유가 이것이다.

주의 헤드라인이 성립하지 않는 자리도 같은 논문 안에 있다. CelebA 위의 VAE 실험에서 논문은 본문에 “데이터를 accumulate 할 때에도 test error 는증가한다” 고 쓰는데 [MCI § 2.3], 같은 그림의 캡션은 “accumulate 가붕괴를 피한다” 고 단정한다 [MCI Fig. 5]. 그리고 같은 실험에서안경·액세서리 같은 부차적 특징이 생성에서 사라진다 — 표 8의 언어로 (0)은 완화되는데 (1)은 일어난다. 재구성은 그 특징을유지하므로 죽는 것은 능력이 아니라 샘플링 분포의 꼬리다. sleep 루프에게는 이쪽이 더 나쁘다. 생성은 재구성이 아니라 샘플링을 쓰기때문이다.

측정의 한계도 기록해 둔다. 오차막대가 없어 모든 곡선이 단일 실행이고, 정량 주장이 전부 held-out loss 이며 downstream 벤치마크가 없다. 실증상한은 126M 파라미터 × 470M 토큰 × 5 라운드인데 논문이 동기론 든 규모는 Llama 의 1.4T-15T 토큰이다. 그리고 그 “실데이터” 인 TinyStories 자체가 GPT-3.5/4 가 생성한 코퍼스다 [MCI § 2.1].

마지막으로 이 corpus 에서 가장 깨끗한 침묵. [MCI] 는 지속학습문헌을 하나도 인용하지 않고, sleep-time compute 쪽의 어느 논문도 [MCI] 를 인용하지 않는다. 두 공동체가 같은 위험을 발견해 서로 다른이름을 붙이고 서로를 0 번 인용한다. 이 다리는 논문이 아니라 이 시리즈가놓는다.

자세한 것은 ch06 § 06.7 과 ch06 § 06.8.

17. 그래서 decode 에서 무엇이 바뀌는가

아무것도 바뀌지 않는다. 그리고 그것이 이 권의 비용이 다른 데숨어 있다는 뜻이다.

sleep 라운드의 산출물은 아키텍처도 파라미터 수도 그대로인 Θ 다. 추가 상태도, KV cache 변화도, 검색 hop 도, 토큰당 오버헤드도 없다. wake 지연 기여 L_w 는 “논문에 없음” 이 아니라 구조적으로 0 이다. 이것이 외부 저장소를 쓰는 경로에 대한 Θ -경로의 구조적 강점이며, 그래서 비용을 다른 데서 찾아야 한다.

시스템 모델링 관점 세 곳에 있다. 첫째, sleep 트래픽은 성격이 반대인 두 작업이다 — 생성은 memory-bandwidth-bound batch decode 이고, 학습은 커지는코퍼스를 순차로 읽는 compute-bound 패스다. 어느 쪽도 지연에 민감하지않고, 압력은 스토리지 용량과 순차 읽기 대역폭에 걸린다. 둘째, 상태 용량 C_{cap} 이 둘로 쪼개진다 — accumulate 에서지속되는 상태는 가중치 delta 가 아니라 보존된 코퍼스다. 가중치 상태는 모델 크기에 고정이지만 데이터 상태는 라운드 수에 선형이고버려지지 않는다. checkpoint 회계만 보는 계획은 이 항을 통째로 놓친다. 셋째, batching 은 살아남는다 — 단, 인구 수준에서만. 라운드당전역 Θ 하나를 만드는 루프는 shared-weight batching 을 깨지않는다. 깨는 것은 문맥마다 다른 가중치를 만드는 사용자 수준 Θ -경로다. 이 구분 없이 “ Θ -경로는 batching 을 깬다” 고쓰면 절반이 틀린다.

마지막으로 모니터링 함정 하나를 다시 못 박는다. 16 절의 VAE 결과는집계 loss 가 유계인 채로 샘플링 분포의 꼬리가 사라질 수 있음을보인다. held-out perplexity 대시보드는 이때 울리지 않는다. 루프의 출력이다음 라운드의 입력이 되는 시스템에서 꼬리 손실은 자기 증폭하므로, 관측지표에 분포 다양성 측정치가 없으면 (1) 번 붕괴는 조용히진행된다.

요약 - state는 KV cache가 담고 있는 token 열, action은 다음 token, reward는 생성이 끝난 뒤 밖에서 도착하는 숫자 하나다. 전이가 결정론적 concat 이라 로봇 RL의 어려움 절반이 없고, 어려움이 전부 “보상을 어디서 구하나”에 몰린다. - policy gradient는 보상을 미분하지 않는다. 미분되는 것은 모델이 그 문장을 낼 확률뿐이며, 그래서 RL은 **미분 불가능한 채점기를 loss로 바꾸는 장치**다. 갱신 단계 자체는 가중치가 붙은 SFT다. - advantage는 보상에서 기준선을 뺀 값이고, 기준선을 만드는 방식이 HBM 예산을 정한다. RLHF의 RL 단계는 모델 넷을 동시에 상주시키며, 이후 계보전체가 그 넷을 줄이는 역사다. DPO는 sampler와 reward model을 함께 지워워크로드를 SFT와 같게 만든다. - RLVR는 보상을 결정론적 프로그램에 맡겨사람 없이 임의 횟수의 보상을 만든다. sleep 라운드가 사람의 처리량에 묶이지 않으려면 이 성질이 필요하다. **보상의 종류가 곧 sleep 라운드가 먹는 하드웨어의 종류다** — 약 6 시간과 약 5분의 차이가 1.4점이다. - reward hacking은 버그가 아니라 대리 최적화의 구조적 귀결이며, 길이 인플레이션이 가장 흔한 형태다. RL 이득 보고에는 표면변량을 prompt 만으로 재현해 보는 통제군이 있어야 한다. - 데이터를 만드는 절차가 일의 대부분을 한다. 증분의 크기 순서는 **형식 > 생성기 품질 > 생성 정책 학습**이고 예산배분도 그 순서다. 필터는 나머지 절반이자 B_s 의 곱셈 인자이며, 분포를 자를 뿐 옮기지 못한다. - model collapse는 네 현상의 이름이다. 번호를 지정하지 않은 붕괴 진술은 결함이다. - 반복 학습의 안정성은 학습 규칙이 아니라 스케줄이 정한다. 라운드당 열화율은 replace에서 상수, replace-multiple에서 $1/k$, accumulate에서 $1/k^2$ 이고, 지속 조건은 그 합의 수렴이다. **데이터를 더 만드는 것은 기제가 아니고, 실데이터를 계속 보유하는 것이 기제다.** - 그 경계는 네 전제 (처음부터 재학습 · 입력 동결 · 정칙화 없음 · 무필터 생성) 위에 서 있고 넷 다 실제 시스템에서 깨진다. $\pi^2/6$ 은 안전 증서가 아니다. - Θ -경로의 wake 지연기여는 구조적으로 0이다. 비용은 sleep 트래픽, 보존 코퍼스, 사용자 수준일때의 batching 붕괴에 있다.

다음 권 예고 — 이 권은 “새 데이터로 배우는 법”과 “그 데이터를 만드는 법”을 다뤘고, 두 계산 모두 **라운드마다 모델을 처음부터 다시 학습한다**는 전제 위에 있었다. 실제 시스템은 그렇게 하지 않는다 — 살아 있는 checkpoint를 라운드마다 증분 갱신한다. 그 순간 이 권의 계산에 없던 실패 양식이 등장한다: 새 것을 배우면서 **이전에 알던 것을 잃는 것**이다. 다음 권이 그 실패와 대응책 (replay, 그리고 뇌가 잠자며 하는 일의 계산적 대응물)을 다룬다.